

A Comparative Study of Software Design Patterns and Anti-Patterns: Active Record versus Repository and Unit of Work

Ioan Teodorescu

EGOV - IB

University Politehnica of Bucharest

Bucharest, Romania

ioan.teodorescu1206@stud.acs.upb.ro

Andreea-Gabriela Matei

EGOV - IB

University Politehnica of Bucharest

Bucharest, Romania

andreea.matei2206@stud.acs.upb.ro

Abstract—Software design patterns provide reusable solutions to recurring problems in software engineering, aiming to improve code quality, maintainability, and performance. However, inappropriate usage of these patterns or the adoption of anti-patterns can negatively affect system scalability and long-term evolution. This paper presents a comparative study between two widely used data access design approaches: the Active Record pattern and the Repository combined with Unit of Work pattern. Using an empirical case study based on a real-world software system, we evaluate these patterns with respect to performance, testability, complexity, and architectural robustness. Furthermore, we analyze common anti-patterns observed in practice, such as the Service Locator, and discuss their consequences. The results highlight that while Active Record offers simplicity and rapid development for small applications, Repository and Unit of Work provide superior scalability, testability, and maintainability for complex systems.

Index Terms—Design Patterns, Anti-patterns, Active Record, Repository Pattern, Unit of Work, Software Architecture, Performance Analysis

I. INTRODUCTION

Software design patterns are general repeatable solutions to common problems in software design. They emerged as a formal concept in the late 20th century, gaining popularity especially after the “Gang of Four” published *Design Patterns: Elements of Reusable Object-Oriented Software* in 1994. Design patterns encapsulate proven development paradigms and best practices, helping engineers avoid subtle issues and write more maintainable code. By providing templates for solving recurring design problems, patterns can speed up development and improve code readability when used appropriately [1].

The concept of anti-patterns is derived from the fact that repeating the same solution on numerous occasions does not justify its efficacy or value. An anti-pattern is defined as an incorrect or ineffective method used repeatedly; however, again, while the method may be “common”, it has proven to be destabilizing and/or ineffective. In 1995, the term was created as a humorous counterpoint to the widely accepted idea of design patterns [2].

The research presented will analyze the contribution of various design patterns to the efficiency and optimization of

software, along with how anti-pattern implementations can lead to negative impacts upon software projects. The current study will review both the Active Record and Repository + Unit of Work architectural patterns as two options for data persistence, assessing their efficiency, discussing their respective benefits and limitations, and describing the impact of related anti-patterns in a case study situation. The aim of the study is to inform software developers about the trade-offs associated with these two patterns, while providing direction regarding how to avoid creating insecure software systems by implementing anti-patterns [2].

II. METHODOLOGY

This paper used a comparative case study approach to understand how effective different design patterns were and how inefficient these patterns were due to anti-patterns. Two very different software design patterns (Active Record, Repository with a Unit of Work) were chosen for comparison because they reflect different techniques for managing object-relational data. The patterns were implemented in the same application, Document Storage System (Docustore), so that an apple-to-apple comparison could be made. Evaluation criteria were defined prior to software development and testing: a) runtime performance metrics (response time, throughput, error rate under load, etc.), b) code quality measures (cyclomatic complexity, coupling, SOLID principle adherence), and c) testability and maintainability measures. Each design pattern selected had multiple anti-patterns related to the approach that warranted further analysis.

The extent of data collection for quantitative analysis was established prior to development. Load test scripts (k6) were used to generate the performance of each pattern when subjected to various load scenarios (smoke tests, sustained load, stress testing, scaling with increasing amounts of data, concurrent user scaling). These test scripts produced the average response time, 95th percentile latency, the number of requests made per second, and the number of failed requests for each of the pattern implementations.

To ensure we have the best quality and maintainable code we collected metrics for the complexity and coupling of all implementations by analyzing the codebases for each one and recording the amount of effort it took to write and execute tests. The Active Record Implementation includes integration tests, which had to be run against a live database and resulted in flaky and/or disabled tests. The Repository & Unit of Work implementation allows fast in-memory unit tests. We kept track of the number of tests created and their execution times and code coverage amounts for both implementations. Each occurrence of any of the well-known anti-patterns found in the code was captured as well, along with the problems that arose as a result of them, (i.e. difficulty tracking dependencies or difficulty testing). During this project, we used software engineering literature and resources from our project repository and classic texts as references to compare our findings to the known best practices and pitfalls of software engineering. [5]

III. ANALYSIS OF DESIGN PATTERNS

A. *Active Record*

Active Record Pattern: An approach to Object Relational Mapping (ORM), the Active Record pattern uses an object to represent each row in a database table directly. Martin Fowler described and named Active Record in 2003. An Active Record design represents both the data (fields mapped to table columns) and includes how those data are manipulated (CRUD functions). Essentially, an Active Record represents "wraps" a row in a database table/view, provides a way to access that row, and applies domain knowledge to it. For instance, an Active Record object will typically contain the insert(), update(), and delete() methods, as well as any other method that operates on its fields and implements the business logic. Saving an object writes it to the database, while loading an object reads it from the database using static methods or ORM-mapped class queries. Although straightforward, and many numerous frameworks/deliberations that have been developed around this concept, this pattern is a foundational model for many frameworks (e.g., the Active Record of Ruby on Rails is an example of Active Record in a convention-over-configuration style of using the Active Record pattern to map Ruby objects to tables). [3] [6] [9]

B. *Repository and Unit of Work*

Repository is a concept originating from Domain-Driven Design established by Eric Evans. Repository is an abstraction layer between the domain and data source (mapping) layers. A Repository provides a set of methods used to retrieve or change domain objects, which allows you to manipulate your data without understanding how it is stored.

The Repository gives you one place to put data access logic; therefore, it ensures that business logic can work with a consistent interface. For instance, if you use the DocumentRepository. Add method, you do not have to worry about how that customer is being stored in the database. All of the specifics related to storing that customer are handled within a

Repository, meaning there will be less impact if the underlying database technology changes. [7]

Repositories are often used in conjunction with the Unit of Work pattern. The Unit of Work serves as a transaction manager that tracks changes to objects during a business operation and commits them as a single transaction. Fowler describes Unit of Work as maintaining a list of all objects changed in a transaction and coordinating the writing out of those changes (and resolution of concurrency issues) in one go. In simpler terms, instead of saving each object to the database one by one (which could be inefficient and risk inconsistent states), the Unit of Work accumulates all intended inserts, updates, and deletes, and flushes them together at a suitable time (e.g. end of a use case), often wrapping them in a database transaction. [8]

IV. EFFICIENCY AND OPTIMIZATION

Defining Efficiency Criteria: There are several aspects to address when measuring the "efficiency" of design patterns, including runtime performance (both speed and resource utilization), scalability under load, and developer efficiency (in regard to maintainability and ease of change); this is typically associated with code quality. In this regard, optimization refers to improving the system with respect to the aforementioned aspects without affecting correctness. We created specific metrics to use when comparing the implementation of Active Record to the combination of Repository and Unit of Work (Repository + UOW). The runtime performance aspect is measured by taking the average response time and the 95th percentile latency of key operations, as well as overall throughput (the total number of requests per second), and the percentage of errors made while processing under differing loads during the testing process. Developer efficiency is measured by analyzing how easy it is to write tests, how clearly the code is written, and, in some cases, various static code metrics, including differences in cyclomatic complexity and coupling between both implementations (the Repository + UOW design exhibited 46% less total complexity and 56% less class coupling in our codebase).

Performance Impact of Each Pattern: The comparative load tests revealed notable differences. Under light load (a "smoke test" of a few concurrent users), both Active Record and Repository+UoW achieved similar throughput with negligible error rates, but the Repository variant was marginally faster – e.g. an average response of 4.95 ms vs 5.18 ms for Active Record (about 4% improvement). This slight edge for Repository became more pronounced under heavier stress. For instance, in a stress test ramping up to 200 concurrent users, the Active Record version's average response time was 10.3 ms compared to 8.4 ms with Repository+UoW – the Repository approach handled the load 18% faster on average. More importantly, reliability under load diverged: Active Record began to show higher error rates (e.g. timeouts or deadlocks) at roughly 1.3% error rate in a high-concurrency scenario, whereas the Repository+UoW version's error rate was around 0.9%, significantly more stable. The critical difference is that

the Repository+UoW design encapsulates database operations in explicit units of work, reducing occurrences of partial failures, while the Active Record design – with each object saving itself independently – could encounter race conditions or missing transactional boundaries leading to errors. It was observed that Active Record had a slight latency advantage in some simple read-heavy scenarios (possibly due to its “simpler” call stack), but those gains were marginal – on the order of a few milliseconds – and were outweighed by its performance degradation and error spikes under high load. Thus, from a pure performance optimization standpoint, the Repository+Unit of Work pattern proved more robust and scalable in our tests, whereas Active Record started strong in trivial cases but struggled to maintain consistency at scale. [5]

Maintainability and Optimization: When considering long-term efficiency (developer productivity, ease of optimizing or extending the code), the Repository+UoW pattern showed clear benefits. Because this pattern enforces a separation of concerns, the code tended to adhere better to fundamental design principles. In our case study, the Repository-based implementation had significantly lower complexity and coupling metrics (as mentioned, complexity was almost half that of the Active Record implementation). This suggests that optimizing or refactoring the Repository code is easier, since changes in database logic stay within repository classes and do not ripple through business logic. It also means new features can be added with less risk of breaking unrelated parts (higher cohesion, lower coupling). On the other hand, the Active Record design intermixes database access with domain logic. This led to some maintainability pain points: for example, the Active Record entities in our project relied on a global Service Locator to fetch the database context and other services, making it hard to trace dependencies and making components implicitly interconnected. Such tight coupling not only complicates understanding the code, but also hampered optimization efforts – even a minor change in how an object is persisted could affect business logic methods and vice versa, making performance tuning risky. Additionally, the Active Record approach exhibited issues like memory leaks and locks due to its use of a shared static database context in tests (multiple tests interfering with a global state). This indicates that certain optimizations (like using connection pooling or lazy loading strategies) are harder to implement safely when state is globally shared, as in the Active Record case.

Testability and Developer Efficiency: Another aspect of efficiency is how quickly and safely a development team can iterate on the code. Here the patterns differed starkly. In our evaluation, the Repository+UoW implementation was highly testable – we could write unit tests for domain logic without a database, using in-memory implementations of repositories, and achieve fast, deterministic tests. In fact, we had 97 unit tests for the Repository pattern code, all of which ran in 0.6 seconds total. These tests gave over 90% branch coverage in the core domain modules, showing that critical business rules were thoroughly checked in isolation. By contrast, the Active

Record implementation’s tests were more like integration tests: out of 85 tests, a significant portion had to be disabled (21% were dropped) because they required a live PostgreSQL database and often failed due to environment issues. The remaining Active Record tests (about 67) ran in 2.2 seconds – slower by factor of 3-4× – and even then, they provided much less coverage of the business logic. The heavy reliance on external DB state made the Active Record tests brittle and slower to execute, reducing confidence in making changes. This clearly impacts developer efficiency: with Active Record, simple refactoring or optimization could not be quickly verified by unit tests, whereas with Repository+UoW, developers could rapidly run a test suite to catch regressions. In summary, considering efficiency broadly (runtime and developer productivity), the Repository+Unit of Work pattern proved more amenable to optimization and sustained maintenance, whereas Active Record, while efficient for quick prototyping, incurred technical debt that made performance tuning and testing more cumbersome over time.

V. COMPARATIVE STUDY

To synthesize the findings, we compare the advantages and disadvantages of the Active Record vs. Repository+Unit of Work patterns across the key criteria:

- **Active Record – Advantages:** Active Record follows a very simple and straightforward workflow – each entity object controls its own persistence. Because of this simplicity, there are fewer layers of abstraction, which helps to expedite the development of simple apps. In specific contexts, Active Record may perform faster than traditional repository types due to the removal of an additional repository. The “convention over configuration” style is similar to how many ORMs (like Rails) are designed and makes it easy for developers to implement basic CRUD operations.
- **Active Record – Disadvantages:** Active Record creates tight coupling between its domain and data schemas. This is particularly evident when one uses a global Service Locator for data access. This results in the business logic and data access being intertwined, violating Single Responsibility Principle and Dependency Inversion Principle. Due to the creation of these intertwining relationships, testing Active Records becomes much more difficult since there are hidden dependencies within the database and any associated environment variables, which often require the use of a live database to properly test. Additionally, in larger systems, Active Record can create scaling problems because making any change to the database schema or creating a more complex transaction will require performing cross-cutting changes in several different classes. Furthermore, within the Active Record pattern, each Active Record class saves its own object independently—therefore, it will not handle the transactional integrity for multiple objects properly due to its lack of a built-in concept for unit-of-work—and this can create issues with data integrity under concurrent access.

- **Repository + Unit of Work – Advantages:** The benefits of separating concerns are clear in the repository pattern because the domain logic does not include any code related to persistence, but rather delegates that responsibility to repository classes. As a result, code becomes more amenable to testing; the domain can be unit tested while using mock or in-memory repositories, allowing for independent testing of actual data access. Additionally, the Unit of Work provides an explicit boundary on transactions so that all changes resulting from a business operation may be committed atomically, thereby providing greater consistency. Lastly, the repository pattern assists with evolving requirements, i.e. as a project becomes increasingly complex, the structure of the repositories and associated data mappers/ORM's provide a means for implementing new features with minimal refactoring.
- **Repository + Unit of Work – Disadvantages:** The cost of using the repository and Unit of Work patterns is that it adds a layer of abstraction, which makes it more complex than if we did not use them, plus it requires additional classes/interfaces, which can seem like wasted effort for simple projects. Using these patterns also adds a level of difficulty for developers who do not have experience with them, along with the need for developers to have the discipline to use them properly. In conclusion, while the Repository and Unit of Work patterns have a higher initial complexity and a lower long-term cost to maintainability, Active Record's low complexity and speed will result in less flexibility over time than the Repository and Unit of Work patterns.

VI. IMPACT OF ANTI-PATTERNS

In the context of our study, we encountered specific anti-pattern manifestations when using the Active Record approach. The most notable was the use of a Service Locator. The Service Locator pattern is widely regarded as an anti-pattern in modern design because it introduces hidden dependencies: objects request needed services from a global registry, which makes the code less explicit and harder to test. In our Active Record implementation, each entity would call a static locator to get database connections or publish events. This led to tightly coupling the domain objects to the infrastructure. The consequences were clearly observed – the code was difficult to unit test, since you had to either set up the global locator with fake services or actually connect to a database for things to work. This anti-pattern also made debugging harder: if something went wrong in an Active Record save operation, the logic was spread between the object and the hidden service, obscuring the root cause. Essentially, the Service Locator usage traded away the dependency injection benefits for a quick access hack, and as predicted by the literature, it hurt maintainability and testability. Another anti-pattern (or at least a code smell) seen in the Active Record design was the violation of fundamental design principles like SRP and DIP, as mentioned earlier. By mixing persistence logic into what should be pure domain logic, the Active Record classes

became too complex and tightly coupled. Such classes are harder to modify (any change can have far-reaching impact) and harder to reuse. They also often require global state (like the aforementioned DB context), which can lead to issues like the memory leaks and concurrency problems we experienced in tests – these are symptomatic of the Shared Mutable State anti-pattern. In summary, adopting the Active Record pattern without caution can inadvertently introduce anti-patterns that undermine the benefits of using a design pattern in the first place. [4]

VII. CONCLUSIONS

The study explored how different software design patterns can affect the effectiveness and sustainment of a system; it also discussed the potential role of design anti-patterns as a warning sign when choosing software design. The report concentrated on comparing the Active Record pattern versus using the Repository and Unit of Work patterns in an equivalent setting. The summary indicates that using the Active Record pattern displayed a fast and easy means of storing information within a database. But the Repository Unit of work method provided a more structured and flexible means of maintaining an entire system over time; the Active Record pattern has certain advantages, such as quicker initial development time and even marginally increased performance in simple cases, but it also has certain disadvantages, including tightly coupling components together and creating increased difficulty with testing for large applications. The Active record pattern often creates an opportunity for anti-patterns, such as using the Service Locator pattern, to arise in large applications. As a result, in large applications, what may appear to be a quick and easy solution can lead to serious long term problems. In contrast, the Repository pattern indicates that although it requires significant time and effort to implement and results in greater complexity than the Active Record pattern, it offers long term benefits in terms of code organization, improved testability, and the ability to handle larger throughput when under load.

REFERENCES

- [1] Wikipedia contributors. Software design pattern. *Wikipedia, The Free Encyclopedia*, Wikimedia Foundation, January 2026. Accessed January 2026.
- [2] Wikipedia contributors. Anti-pattern. *Wikipedia, The Free Encyclopedia*, Wikimedia Foundation, January 2026. Accessed January 2026.
- [3] Wikipedia contributors. Active record pattern. *Wikipedia, The Free Encyclopedia*, Wikimedia Foundation, January 2026. Accessed January 2026.
- [4] Wikipedia contributors. Service locator pattern. *Wikipedia, The Free Encyclopedia*, Wikimedia Foundation, January 2026. Accessed January 2026.
- [5] Andreea-G. Matei and Ioan Teodorescu. DocuStore Active Record vs Repository+UoW – Comparative Analysis. SASPS-Final Project README, GitHub, 2026.
- [6] Martin Fowler. Active Record. In *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2003. Available online at martinfowler.com/eaCatalog.
- [7] Martin Fowler. Repository. In *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2003. Available online at martinfowler.com/eaCatalog.

- [8] Martin Fowler. Unit of Work. In *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2003. Available online at martinfowler.com/eaCatalog.
- [9] Rails Guides. What is Active Record (ORM)? *Ruby on Rails v6.1 Documentation*, 2019.